第二章 WebDriver API

2.1 WebDriver API介绍

2.2 浏览器操作

2.3 界面元素操作

2.4 元素定位

2.5 获取界面元素内容

2.6 键盘

2.7 鼠标

2.8 多窗口

2.9 多表单

2.10 弹出框

2.11 下拉框

2.12 等待

2.13 文件上传

2.14 截图

2.15 验证码

汇聚智慧，动力无限

2.1 WebDriver API介绍

API（Application Programming Interface,应用程序编程接口）是一些预先定义的函数，目的是提供应用程序与开发人员基于某软件或硬件得以访问一组例程的能力，而又无需访问源码，或理解内部工作机制的细节。

基于Python的Webdriver API是Python语言通过调用API来操作浏览器。API库针对多种语言实现，所以可以支持Python、Java、Ruby、C#等语言调用。

汇聚智慧，动力无限

2.2 浏览器操作

- get(url)：打开指定页面
- set_window_size(x,y)　：设置窗口大小
- maximize_window()：放大浏览器
- forward()：前进
- back()：后退
- refresh()：页面刷新
- close() 关闭当前窗口
- quit()退出浏览器

汇聚智慧，动力无限

2.2 浏览器操作

```
import time
from selenium import webdriver

#创建实例
driver = webdriver.Chrome()
#打开网页
driver.get("http://www.baidu.com")
#浏览器窗口设置为: 600*800
driver.set_window_size(600, 800)
time.sleep(3)
#浏览器窗口最大化
driver.maximize_window()
time.sleep(3)
#打开百度百科
driver.get("http://baike.baidu.com")
time.sleep(3)
#返回百度页面
driver.back()
time.sleep(3)
#前进到百度百科
driver.forward()
time.sleep(3)
#刷新页面
driver.refresh()
#退出浏览器
driver.close()
```

汇聚智慧，动力无限

2.3 界面元素操作

- click() ：点击界面元素按钮，适用于任何元素
- clear() ：清除input元素内容
- send_keys() ：发送文字，适用于input元素
- submit()：用于提交表单、信息，相当于回车操作或者点击登录按键，适用于form元素。

汇聚智慧，动力无限

2.4 元素定位

常见的元素定位方式：

元素的定位与操作是自动化测试的核心部分，其中操作又是建立在定位的基础上的，因此元素定位就显得比较重要。

一个元素的定位，他会有各种的特征（属性），比如我们可以通过一个人的身份证号码、姓名或者他的地址找到这个人。那么元素也是有类似的属性的，我们可以通过这个唯一区别于其他元素的属性来定位这个元素。当然，除了要操作的元素时需要定位，有的时候我们需要获得元素的具体text的值或者属性值时，也需要先定位元素，再进行进一步的定位。

2.4 元素定位

- **id**：find_element_by_id()
- **name**：find_element_by_name()
- **class name**：find_element_by_class_name()
- **link text**：find_element_by_link_text()
- partial link text:find_element_by_partial_link_text()
- **tag name**：find_element_by_tag_name()
- **xpath**：find_element_by_xpath()
- **css**：find_element_by_css_selector()

汇聚智慧，动力无限

2.4 元素定位

◆使用id定位：id是在DOM中唯一标志这个元素的属性，没有找到selenium.common.exceptions.NoSuchElementException

```
import time
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://www.baidu.com")
#输入框中输入'selenium'
driver.find_element_by_name("wd").send_keys("selenium")
time.sleep(3)
#点击'百度一下'
driver.find_element_by_id("su").click()
time.sleep(3)
driver.close()
```

汇聚智慧，动力无限

2.5 获取界面元素内容

- title：获取页面标题
- current_url：获取当前页面url地址

- size：获取元素的尺寸
- text：获取元素的文本
- get_attribute:获取元素属性值
  - ✓ 某个属性的值
  - ✓ ele.get_attribute('href')

汇聚智慧，动力无限

2.4 元素定位

- 一个web元素

```
<input name=" cheese"  type=" text" />
```

- 返回的是第一个找到的元素

```
chees = driver.find_element_by_name( “cheese” )
```

- 返回所有元素

```
cheeses = driver.find_elements_by_name( “cheese” )
```

如果找不到，返回空列表，不抛异常

汇聚智慧，动力无限

2.4 元素定位

使用class name定位：为了弥补前端开发人员没有使用id和name属性时的情况，可以使用class属性来进行定位。

```
# coding=utf8
from selenium import webdriver


driver = webdriver.Chrome(r"C:\tools\webdriver\chromedriver.exe")

driver.get(r'file:///C:\test\课程内容\Selenium\FileRecv\samples_selenium\wd\lesso

ele = driver.find_element_by_class_name("cheese")
# for ele in eles:
#     print('--------------------')
#     print(ele.get_attribute('outerHTML'))
#     print(ele.text)
print(ele.get_attribute('outerHTML'))


driver.quit()
```

汇聚智慧，动力无限

2.4 元素定位

- 可以根据tag名定位
- 假设html中有如下片段

```
<footer>
        <div>
            <p>test1</p>
        </div>
    </footer>
```

- 可以

```
footer= driver.find_element_by_tag_name("footer")
```

汇聚智慧，动力无限

2.4 元素定位

使用link text、partial link text定位：有时需要操作的元素时一个文字链接，可以通过link text或者 partial link text进行定位操作。

```
# coding=utf8
import ...


driver = webdriver.Chrome()

driver.get('http://www.baidu.com') # 打开网址


# ----------------------------------
#定位元素
link = driver.find_element_by_partial_link_text(u"新")
alink = 'http://news.baidu.com/'
if link.get_attribute('href') == alink:
    print('测试通过')
else:
    print('测试不通过')
#link.click()
```

汇聚智慧，动力无限

多种技术实现目的

◆例子

访问天气查询网站，查询四川省的天气

http://www.weather.com.cn/html/province/sichuan.shtml

获取四川省所有城市的天气，并找出其中每天最低气温最低的城市，显示出来。比如：

温度最低为12℃，城市有成都　眉山

汇聚智慧，动力无限

2.4 元素定位

- 根据tag名

  p { color: red;}

- 根据ID

  #food {color: red;}

- 根据class

  .vegetable {color: red;}

- 根据tag名和class组合（如果多个）

  span.vegetable {color: red;}

汇聚智慧，动力无限

2.4 元素定位

- 修改代码，用css选择器选择web元素

  find_element_by_css_selector

  find_elements_by_css_selector

- 根据tag名

  driver.find_elements_by_css_selector('p')

- 根据id

  driver.find_elements_by_css_selector('#food')

- 根据class

  driver.find_elements_by_css_selector('.vegetable')

```
<span class="vegetable good">黄瓜</span>
```

汇聚智慧，动力无限

2.4 元素定位

- 后代选择器的选择元素内部的元素
  - 语法

    <s1> <s2>

    - 选择s1元素里面所有的s2元素
    - 可以是直接子节点，也可以不是
  - 比如
    - # choose_car option
    - footer p
  - 可以是很多级

    ul ol li em {color: blue;}

汇聚智慧，动力无限

2.4 元素定位

- 子选择器选择元素的子元素
  - 和后代选择器不同：#choose_car option
  - 比如
    - #choose_car > option
    - footer > p
  - 可以是很多级
    - ul > ol > li >em

汇聚智慧，动力无限

2.4 元素定位

兄弟节点：

- 选择紧接在另一个元素后的元素，二者有相同的父元素
  - 比如
    - #food + div
    - #many > div > p.special + p
- 选择在另一个元素后的元素，二者有相同的父元素
  - 比如
    - # food ~ div

汇聚智慧，动力无限

2.4 元素定位

- 可以根据元素的属性值来选择元素
  - 比如
    - *[style]
    - p[spec=len2]
    - p[spec='len2 len3']
    - p[spec*='len2']　　#包含
    - p[spec^='len2']　　#开头　结尾
    - p[spec$='len2']
    - p[class=special][name=p1]
    - http://www.runoob.com/cssref/css-selectors.html

汇聚智慧，动力无限

2.4 元素定位

[attribute^=value]	a[src^="https"]	选择其 src 属性值以 "https" 开头的每个 <a> 元素。
[attribute$=value]	a[src$=".pdf"]	选择其 src 属性以 ".pdf" 结尾的所有 <a> 元素。
[attribute*=value]	a[src*="abc"]	选择其 src 属性中包含 "abc" 子串的每个 <a> 元素。
:first-of-type	p:first-of-type	选择属于其父元素的首个 <p> 元素的每个 <p> 元素。
:last-of-type	p:last-of-type	选择属于其父元素的最后 <p> 元素的每个 <p> 元素。
:only-of-type	p:only-of-type	选择属于其父元素唯一的 <p> 元素的每个 <p> 元素。
:only-child	p:only-child	选择属于其父元素的唯一子元素的每个 <p> 元素。
:nth-child(n)	p:nth-child(2)	选择属于其父元素的第二个子元素的每个 <p> 元素。

:nth-of-type(n) p:nth-of-type(3) 选择属于其父元素的第3个<p>元素

汇聚智慧，动力无限

2.4 元素定位

```
import time
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://zhidao.baidu.com")

#搜索“selenium”
driver.find_element_by_css_selector(".hdi").send_keys("selenium")
driver.find_element_by_css_selector("#search-btn").click()
time.sleep(3)
#点击其中一条搜索
driver.find_element_by_css_selector(".title > a:nth-child(1)").click()
time.sleep(3)

driver.quit()
```

汇聚智慧，动力无限

2.4 元素定位

- http://www.51job.com
- 获取搜索关键词 python， 地区选择 杭州（注意，所在地已经选中，要取消），搜索最新发布的职位， 抓取页面信息。 得到如下的格式化信息

- Python开发工程师 | 杭州纳帕科技有限公司 | 杭州 | 0.8-1.6万/月 | 04-27
- Python高级开发工程师 | 中浙信科技咨询有限公司 | 杭州 | 1-1.5万/月 | 04-27
- 高级Python开发工程师 | 杭州新思维计算机有限公司 | 杭州-西湖区 | 1-1.5万/月 | 04-27

2.4 元素定位

使用xpath定位：

前面几种元素都无法定位的情况下，才使用xpath路径定位。

xpath节点定位方式：

表达式	描述
nodename	选取此节点的所有子节点。
/	从根节点选取。
//	从匹配选择的当前节点选择文档中的节点，而不考虑它们的位置。
.	选取当前节点。
..	选取当前节点的父节点。
@	选取属性。

路径表达式	结果
bookstore	选取 bookstore 元素的所有子节点。
/bookstore	选取根元素 bookstore。 注释：假如路径起始于正斜杠(/)，则此路径始终代表到某元素的绝对路径！
bookstore/book	选取属于 bookstore 的子元素的所有 book 元素。
//book	选取所有 book 子元素，而不管它们在文档中的位置。
bookstore//book	选择属于 bookstore 元素的后代的所有 book 元素，而不管它们位于 bookstore 之下的什么位置。
//@lang	选取名为 lang 的所有属性。

汇聚智慧，动力无限

2.4 元素定位

xpath的常用语法：

- 通过绝对路径定位元素（不推荐使用）：

```
find_element_by_xpath("/html/body/div/form/input");
```

- 通过相对路径定位元素：

```
find_element_by_xpath("//div/input");
```

- 使用索引定位元素：

```
find_element_by_xpath("//input[4]");
```

汇聚智慧，动力无限

2.4 元素定位

xpath的常用语法：

- 使用xpath及属性值定位元素：

  find_element_by_xpath("//input[@id='kw']");

- 部分属性值匹配：

  find_element_by_xpath("//*[contains(@href, 'logout')]")匹配包含属性的值

  find_element_by_xpath(" //*[starts-with(@id,'test_value')]")—匹配开始字段，ends-with方法不可用

汇聚智慧，动力无限

2.4 元素定位

- 参考http://www.w3school.com.cn/xpath/xpath_axes.asp
- 相邻兄弟选择器
  - following-sibling

    //*[@id="food"]/p/following-sibling::*
  - preceding-sibling

    //*[@id="food"]/p/preceding-sibling::span[2]
- xpath擅长的
  - 选择父节点 ..符号

    //*[@class="special"]/..
  - position函数，结合比较操作符

汇聚智慧，动力无限

2.6 键盘

操作键盘使用selenium.webdriver.common.keys import Keys的Keys类,一般send_keys函数联用，常见的方法：

- send_keys(Keys.BACK_SPACE) 删除键（BackSpace）
- send_keys(Keys.SPACE) 空格键(Space)
- send_keys(Keys.TAB) 制表键(Tab)
- send_keys(Keys.ESCAPE) 回退键（Esc）
- send_keys(Keys.ENTER) 回车键（Enter）
- send_keys(Keys.CONTROL,'a') 全选（Ctrl+A）
- send_keys(Keys.CONTROL,'c') 复制（Ctrl+C）

2.6 键盘

- send_keys(Keys.CONTROL,'x') 剪切（Ctrl+X）
- send_keys(Keys.CONTROL,'v') 粘贴（Ctrl+V）

```
import time
from selenium import webdriver
from selenium.webdriver.common.keys import Keys

driver = webdriver.Chrome()
driver.get("https://www.baidu.com")
#输入框中输入'selenium'
driver.find_element_by_name("wd").send_keys("selenium")
time.sleep(3)
#输入框中ctrl+a
driver.find_element_by_name("wd").send_keys(Keys.CONTROL, "a")
time.sleep(3)
#输入框中输入'汇智动力'
driver.find_element_by_name("wd").send_keys("汇智动力")
time.sleep(3)
driver.close()
```

2.7 鼠标

常见鼠标操作包含在selenium.webdriver.common.action_chains的ActionChains类中，常用的方法：

- context_click() 右击
- double_click() 双击
- click_and_hold()　左击悬停
- move_to_element()　鼠标悬停
- drag_and_drop() 拖动
- perform() 执行ActionChains中存储的行为
- 页面上查看悬停元素：setTimeout(function(){debugger;},5000)

汇聚智慧，动力无限

2.7 鼠标

```
import time
from selenium import webdriver
from selenium.webdriver.common.action_chains import ActionChains

driver = webdriver.Chrome()
driver.get("https://www.baidu.com")
#定位到"设置"元素悬停
hold_ele = driver.find_element_by_link_text("设置")
ActionChains(driver).move_to_element(hold_ele).perform()
time.sleep(3)
#点击"搜索设置"
driver.find_element_by_link_text("搜索设置").click()
time.sleep(3)
driver.close()
```

2.8 多窗口

在页面操作中点击链接有时会弹出一个新的浏览器窗口，这就涉及多窗口的切换操作。

- driver.current_window_handle　获取当前窗口handle
- driver.window_handles　获取所有窗口handle
- driver.switch_to.window(handle)　切换窗口

汇聚智慧，动力无限

2.8 多窗口

```
import time
from selenium import webdriver

driver = webdriver.Chrome()
driver.get("https://zhidao.baidu.com")
#获取当前窗口的句柄（窗口的唯一标识用句柄表示）
first_window = driver.current_window_handle
print(first_window)
#搜索“selenium”
driver.find_element_by_id("kw").send_keys("selenium")
driver.find_element_by_id("search-btn").click()
time.sleep(3)
#点击其中一条搜索
driver.find_element_by_css_selector(".title > a:nth-child(1)").click()
#查看当前页面标题
print(driver.title)
#切换窗口
windows = driver.window_handles
print(windows)
for i in windows:
    if i == first_window:
        continue
    else:
        driver.switch_to.window(i)
        print(driver.title)
time.sleep(3)
driver.quit()
```

汇聚智慧，动力无限

2.9 嵌套页面

Web页面中经常用到Frame/Iframe表单嵌套页面，使用Frame重载页面时不需要重载整个页面，只需要重载页面中的一个框架页(减少了数据的传输，加快了网页下载速度)。

WebDriver只能在一个页面上对元素识别与定位，如果跨Frame定位元素是无法直接定位的，必须先切换到该元素所在的Frame才能定位。

- switch_to.frame()　切换到指定Frame
- switch_to.parent_frame() 切换到上一层frame
- switch_to.default_content() 跳到最外层frame

https://music.163.com/#/discover/toplist

汇聚智慧，动力无限

2.9 嵌套页面

```python
# coding:utf8
from selenium import webdriver

driver = webdriver.Chrome(r"d:\tools\webdrivers\chromedriver.exe")
driver.implicitly_wait(10)

# 抓取排行榜信息
driver.get('https://music.163.com/#/discover/toplist')

driver.switch_to.frame(driver.find_element_by_id("g_iframe"))

div = driver.find_element_by_id("song-list-pre-cache")
print(div.text)

driver.switch_to.default_content()
ele = driver.find_element_by_id('g-topbar')
print(ele.text)

driver.quit()
```

2.10 弹出框

对js使用的alert、confirm 以及 prompt定位也是项目中常见的，比如弹出提示框“确定”等。要定位这类提示框具体思路是switch_to_alert()方法定位alert/confirm/prompt，然后使用text/accept/dismiss/send_keys这一系列动作。

- driver.switch_to.alert.accept() #点击ok
- driver.switch_to.alert.text #得到对话框内容
- driver.switch_to.alert.dismiss() #点击Cancel
- driver.switch_to.alert.send_keys() #输入内容

汇聚智慧，动力无限

2.10 弹出框

```
# ------------------------------------
driver.find_element_by_id('b3').click()

time.sleep(1)

alert = driver.switch_to.alert
print(alert.text)
#alert.dismiss()
alert.send_keys('今天心情不错')
alert.accept()

driver.find_element_by_id('other').click()

# ------------------------------------
input('press any key to quit...')
driver.quit()    # 浏览器退出
```

汇聚智慧，动力无限

2.11 下拉框

常见下拉框操作包含在selenium.webdriver.support.select的Select 类中，常用的方法：

- select_by_value('value')　通过value值选择
- select_by_visible_text('text')　通过文本选择
- select_by_index(index)　通过索引选择，索引从0开始

汇聚智慧，动力无限

2.11 下拉框

```
from selenium import webdriver
from time import sleep
from selenium.webdriver.support.select import Select

driver = webdriver.Chrome()
driver.get("http://tieba.baidu.com/f/search/adv?red_tag=v1695260305")
#查找下拉框元素
select_ele = driver.find_element_by_name("sm")
#创建Select类的对象
select_ob = Select(select_ele)
#根据'value'选择
select_ob.select_by_value("0")
sleep(2)
#根据索引选择
select_ob.select_by_index(0)
sleep(2)
#根据文本选择
select_ob.select_by_visible_text("按相关性排序")
sleep(2)
driver.quit()
```

汇聚智慧，动力无限

2.12 等待

等待分为隐式等待及显式等待。隐式等待：在设定的时间t内等到页面上的元素加载完成。若在时间t内元素加载完成则剩下的时间不需要再等待，若时间t内元素还未被加载则系统报错。隐式等待是全局的，作用在整个脚本上，所以在一个脚本中只需要设置一次即可。显式等待：在设定的时间内，按照固定的时间间隔扫描指定元素是否加载完成，加载完成则剩下的时间不再等待，代码继续往下执行，如果在时间t内没有加载完成则系统报错。

注意：隐式等待和显式等待可以设置，等待的最长时间取两者之中的大者。

- implicitly_wait(max_time)　　隐式等待时间
- WebDriverWait(driver, max_time, check_time).until(expected_conditions.presence_of_element_located(locator))

max_time最长等待时间，check_time每隔xx时间判断，locator判断的元素

汇聚智慧，动力无限

2.12 等待

```
import time
from selenium import webdriver
from selenium.webdriver.support.wait import WebDriverWait as WDW
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
#设置等待加载时长
driver.implicitly_wait(1)
driver.get("http://www.taobao.com")

#设置一定时间内，每隔一段时间检测元素是否存在
WDW(driver, 10, 0.2).until(EC.presence_of_element_located((By.CSS_SELECTOR,
    "div.J_Ald_201602246:nth-child(2) > a:nth-child(3) > img:nth-child(1)")))
#强制等待
time.sleep(3)
driver.close()
```

汇聚智慧，动力无限

2.13 上传文件

- 普通上传：普通的附件上传是将本地文件的路径作为一个值放在input 标签中，通过form 表单将这个值提交给服务器。可以通过send_keys（"文件路径"）直接上传。
- 插件上传：一般是指基于Flash、JavaScript 或Ajax 等技术所实现的上传功能，通过AutoIt工具辅助。

汇聚智慧，动力无限

2.15 上传文件

al.py uploadfile.py

```
import time

driver = webdriver.Chrome()
driver.implicitly_wait(10)


driver.get('https://www.baidu.com')
#点击图片
driver.find_element_by_class_name("soutu-btn").click()
time.sleep(3)
file_path=r'C:\test\page1.jpg'
#点击本地上传图片
driver.find_element_by_class_name("upload-pic").send_keys(file_path)
time.sleep(10)

input('...')
```

汇聚智慧，动力无限

2.16 截图

一般在执行用例时，如果用例失败，需要将当前窗口截图保存

方法：get_screenshot_as_file('imgname')

```
from selenium import webdriver

dr = webdriver.Chrome()
dr.get("https://www.baidu.com")
dr.get_screenshot_as_file('imgname.png')

dr.close()
```

2.17 验证码

验证码操作方法：

1. 去掉验证码
2. 设置万能码
3. 验证码识别技术（OCR）（例如Python-tesseract识别图片验证码）

https://www.cnblogs.com/landhu/p/4968577.html

汇聚智慧，动力无限

2.18 非input标签输入文本

```
#利用javaScript来插入内容
js = 'document.getElementById("xxx").innerHTML="<p>this is test!</p>" '
driver.execute_script(js)
```

汇聚智慧，动力无限